

LINUX SECURITY

Personal Study Notes

A detailed personal reference guide covering physical security, user management, file permissions, kernel hardening, network defence, incident response, and more.

Table of Contents

1. Introduction & Overview
2. Physical Security
3. Local User & Account Security
4. File System Security & Permissions
5. Password Security & Encryption
6. Kernel Security
7. Network Security
8. Security Preparation Before Going Online
9. Incident Response — Break-In Handling
10. Closing the Hole & Damage Assessment
11. Backups — Your Safety Net
12. Tracking Down the Intruder
13. Security Resources & Glossary
14. FAQs & Conclusion

1. Introduction & Overview

Linux powers a huge portion of the world's servers, cloud infrastructure, embedded systems, and enterprise workloads. Its open-source nature means transparent code — but it also means anyone, including attackers, can read that code looking for weaknesses. Understanding Linux security is therefore not optional for anyone responsible for a Linux machine.

Security is best thought of not as a product you purchase but as a **continuous process**. New vulnerabilities are discovered daily; the threat landscape shifts constantly. A system that was perfectly hardened six months ago may be exposed today simply because a dependency was not updated. This guide exists to give you a solid mental model of the major attack surfaces in a Linux system and the practical controls available for each.

■ The CIA Triad — Why Security Matters

Every security decision can be mapped back to three fundamental goals:

- **Confidentiality** — Only authorised parties can read data. Example: student records must not be visible to unauthenticated users.
- **Integrity** — Data cannot be altered without detection. Example: a tampered system binary is a huge red flag.
- **Availability** — Systems must remain usable by legitimate users. Example: a DDoS attack that knocks your web server offline violates availability.

■ Defence-in-Depth

No single control is bulletproof. A firewall alone does not stop an attacker who already has a user account. Strong passwords alone cannot prevent a physical theft. The principle of **defence-in-depth** says: layer multiple independent controls so that the failure of any one layer does not result in a total compromise. Think of it as the multiple locked doors, CCTV cameras, and guards in a bank vault — each layer slows an attacker and gives you more opportunity to detect and respond.

■ Key Mindset

Assume breach. Always ask: 'If an attacker gets past this control, what is the next line of defence?' Designing systems with this question in mind produces far more resilient architectures.

2. Physical Security

All software-level security controls collapse the moment an attacker gains unrestricted physical access to a machine. They can boot from a USB drive, bypass full-disk encryption on an unlocked system, replace hardware, install keyloggers, or simply walk out the door with the disk.

Physical security is therefore the first and most fundamental layer.

■ Server Room Controls

- **Lock server rooms and data centres** with key-card or biometric access. Maintain an entry log — who entered and when.
- **CCTV cameras** at entrances, exits, and server aisles create a deterrent and provide evidence after an incident.
- **Principle of least access** — only personnel whose job requires it should have physical access. Contractors and visitors must be escorted.
- **Cable locks** on workstations in open office environments prevent opportunistic hardware theft.
- **BIOS/UEFI passwords** prevent an attacker from changing boot order to boot from removable media.
- **Disable unused ports** — USB, FireWire, Thunderbolt. A plugged-in USB stick can deliver malware in seconds.

■ Workstation & Laptop Considerations

Laptops are at especially high risk because they leave the building. Full-disk encryption (via **LUKS** on Linux) ensures that even if a laptop is stolen, the data remains unreadable without the passphrase. Enable a BIOS boot password and configure the system to lock automatically after a short idle period.

```
# Enable full-disk encryption during OS installation (LUKS) # or encrypt a
partition later: cryptsetup luksFormat /dev/sdb1 cryptsetup luksOpen
/dev/sdb1 secure_vol mkfs.ext4 /dev/mapper/secure_vol
```

■ *Physical security is often overlooked in documentation because it feels 'obvious,' but it accounts for a significant share of real-world breaches — particularly insider threats.*

3. Local User & Account Security

Linux is inherently a multi-user operating system. Every running process belongs to a user, and the permissions that process has are determined by that user's privileges. Getting user and group management right is therefore foundational to the entire security posture of the system.

■ Principle of Least Privilege

Each user account — and each service account — should have only the permissions it needs to do its job, and nothing more. A web server does not need root. A developer does not need access to the payroll database. Granting excessive permissions creates unnecessary risk: if that account is compromised, the attacker inherits all of its privileges.

■ User Account Management

- Create individual named accounts for each person — never share accounts.
- Disable or delete accounts of users who no longer need access (ex-employees, decommissioned services).
- Use **sudo** for administrative tasks rather than logging in as root directly. Sudo logs every command with a timestamp and the invoking user.
- Lock accounts temporarily with **usermod -L username** and unlock with **-U**.

```
# Create a new user useradd -m -s /bin/bash alice passwd alice # Add user
to sudo group (Debian/Ubuntu) usermod -aG sudo alice # Lock an account
usermod -L bob # List all users cat /etc/passwd # View sudo access cat
/etc/sudoers # or use visudo
```

■ Monitoring User Activity

- **last** — shows recent login history.
- **who / w** — shows currently logged-in users.
- **auditd** — comprehensive kernel-level audit framework; logs file access, command execution, and system calls.
- Review **/var/log/auth.log** (Debian) or **/var/log/secure** (RHEL) regularly for failed logins and sudo usage.

```
# Check recent logins last -n 20 # Watch who is online right now w # Search
for failed SSH login attempts grep 'Failed password' /var/log/auth.log |
tail -30
```

■ Service Accounts

Always create dedicated, non-login service accounts for daemons (e.g., www-data for nginx). Set their shell to /usr/sbin/nologin and disable password login. This contains any compromise of that service to just its restricted account.

4. File System Security & Permissions

Linux file permissions are a powerful access control mechanism that predates more modern models. Understanding them deeply — including the often-misunderstood sticky bit and SUID/SGID — is essential for hardening any system.

■ The Permission Model

Every file and directory has three permission sets: **owner**, **group**, and **others**. Each set contains read (r=4), write (w=2), and execute (x=1) bits.

```
# View permissions ls -la /etc/passwd # Set permissions (owner rwx, group
rx, others rx) chmod 755 script.sh # Change ownership chown
alice:developers project/ # Recursively change ownership chown -R www-
data:www-data /var/www/html/
```

■ Special Bits: SUID, SGID, Sticky

- **SUID (4xxx)** — When set on an executable, it runs with the file owner's privileges, not the caller's. Dangerous if applied carelessly. Example: `/usr/bin/passwd` is SUID root so users can change their own password.
- **SGID (2xxx)** — On a directory, new files inherit the directory's group, which is useful for shared project folders.
- **Sticky bit (1xxx)** — On a directory (like `/tmp`), users can only delete files they own, even if they have write permission on the directory.

```
# Find all SUID files — audit these! find / -perm -4000 -type f 2>/dev/null
# Set sticky bit on shared directory chmod +t /shared
```

■ Sensitive Files to Protect

- **/etc/passwd** — readable by all, but should never be writable except by root.
- **/etc/shadow** — hashed passwords. Must be readable only by root (mode 640 or 000).
- **/etc/sudoers** — only editable via **visudo**, which validates syntax.
- **/root** — home directory of root; should be mode 700.
- **SSL private keys** — typically mode 600, owned by root or the service user.

■ Access Control Lists (ACLs)

Standard Unix permissions are sometimes too coarse. ACLs allow per-user, per-group fine-grained permissions on top of the standard model.

```
# Install acl tools apt install acl # Grant user bob read access to a file
setfacl -m u:bob:r file.txt # View ACLs getfacl file.txt
```

5. Password Security & Encryption

Passwords remain the primary authentication mechanism for most systems. Despite decades of warnings, weak passwords continue to be one of the leading causes of security breaches.

Encryption complements authentication by ensuring that data is unreadable even when stolen.

■ What Makes a Password Strong?

- **Length** — The single biggest factor. A random 20-character passphrase is vastly harder to crack than a complex 8-character password.
- **Entropy** — Avoid dictionary words, names, dates, keyboard patterns (qwerty), and predictable substitutions (p@ssw0rd).
- **Uniqueness** — Each account must have its own password. Credential stuffing attacks rely on password reuse.
- **Storage** — Use a password manager (Bitwarden, KeePassXC). Never store passwords in plain text.

■ Enforcing Password Policies with PAM

Linux Pluggable Authentication Modules (PAM) let you enforce minimum length, complexity, history, and expiry centrally.

```
# Install password quality library apt install libpam-pwquality # Edit
/etc/security/pwquality.conf minlen = 14 dcredit = -1 # require at least 1
digit ucredit = -1 # at least 1 uppercase lcredit = -1 # at least 1
lowercase ocredit = -1 # at least 1 special char # Set password expiry (90
days max, warn 14 days before) chage -M 90 -W 14 alice
```

■ SSH Key Authentication — Prefer Over Passwords

For remote access, SSH key pairs are dramatically more secure than passwords. A 4096-bit RSA or Ed25519 key is computationally infeasible to brute-force.

```
# Generate an Ed25519 key pair (recommended) ssh-keygen -t ed25519 -C
'myserver' # Copy public key to server ssh-copy-id -i ~/.ssh/id_ed25519.pub
user@server # Disable password login in /etc/ssh/sshd_config: #
PasswordAuthentication no # PermitRootLogin no systemctl restart sshd
```

■ Encryption at Rest and in Transit

- **LUKS** — Full-disk encryption for Linux. Integrated into the kernel via dm-crypt.
- **GnuPG (GPG)** — Encrypt individual files or email attachments.
- **TLS/SSL** — All network services handling sensitive data must use TLS. Obtain free certificates from Let's Encrypt.
- **SSH** — Encrypted remote shell. Always prefer SSH over Telnet or unencrypted FTP.

■ Multi-Factor Authentication (MFA)

Enable MFA wherever possible — especially for SSH, web admin panels, and VPNs. The Google Authenticator PAM module or FIDO2/U2F hardware tokens add a second factor that dramatically reduces risk even if a password is compromised.

6. Kernel Security

The Linux kernel is the software layer that sits between applications and hardware. It manages memory, processes, devices, and networking. Because the kernel runs with the highest privilege level (ring 0), a kernel vulnerability can give an attacker complete control over the entire system. Kernel security is therefore non-negotiable.

■ Keep the Kernel Updated

Kernel updates often contain critical security patches. A rule of thumb: always run the latest stable kernel for your distribution. Set up unattended upgrades for security patches.

```
# Debian/Ubuntu - enable automatic security updates apt install unattended-
upgrades dpkg-reconfigure unattended-upgrades # Check current kernel
version uname -r # List installed kernels dpkg --get-selections | grep linux-image
```

■ Mandatory Access Control — SELinux and AppArmor

Standard Unix DAC (Discretionary Access Control) lets the file owner decide who has access.

MAC (Mandatory Access Control) adds a system-wide policy enforced by the kernel that cannot be overridden even by root.

- **SELinux** (Security-Enhanced Linux) — used in RHEL/CentOS/Fedora. Every process and file gets a label; the policy defines which labels can interact.
- **AppArmor** — used in Ubuntu/Debian. Profile-based; easier to configure than SELinux but less granular.

```
# AppArmor status aa-status # Enforce a profile aa-enforce
/etc/apparmor.d/usr.sbin.nginx # SELinux status getenforce # Temporarily
set SELinux to permissive (log but don't block) setenforce 0
```

■ Kernel Hardening with sysctl

Many kernel security settings are tunable at runtime via **sysctl**. Add these to `/etc/sysctl.d/99-security.conf` and run `sysctl -p`.

```
# Prevent IP spoofing net.ipv4.conf.all.rp_filter = 1 # Disable ICMP
redirect acceptance net.ipv4.conf.all.accept_redirects = 0 # Enable SYN
flood protection net.ipv4.tcp_syncookies = 1 # Restrict dmesg to root
kernel.dmesg_restrict = 1 # Disable core dumps for SUID programs
fs.suid_dumpable = 0 # Apply changes sysctl -p /etc/sysctl.d/99-
security.conf
```

■ Disabling Unnecessary Kernel Modules

Kernel modules extend functionality but each one is additional attack surface. If your server does not need USB storage, Bluetooth, or the DCCP protocol, blacklist those modules.

```
# Blacklist a module (create file in /etc/modprobe.d/) echo 'blacklist  
usb-storage' > /etc/modprobe.d/blacklist-usb.conf echo 'blacklist dccp' >>  
/etc/modprobe.d/blacklist-protocols.conf # Verify module is not loaded  
lsmod | grep usb_storage
```

7. Network Security

Most compromises today arrive over the network. A server exposed to the internet without proper controls is typically attacked within minutes of being online. Network security is a broad topic; here we cover firewalling, VPNs, and intrusion detection.

■ Firewalls — UFW and iptables

UFW (Uncomplicated Firewall) is a frontend for iptables that makes common rules much simpler to write. The golden rule: **default deny** inbound, then open only what is needed.

```
# Enable UFW with default deny ufw default deny incoming ufw default allow outgoing ufw enable # Allow SSH (critical — do this BEFORE enabling if remote) ufw allow 22/tcp # Allow HTTP and HTTPS ufw allow 80/tcp ufw allow 443/tcp # Allow from specific IP only ufw allow from 192.168.1.0/24 to any port 5432 # Check status ufw status verbose
```

■ Intrusion Detection — fail2ban

fail2ban monitors log files for repeated authentication failures and automatically adds firewall rules to block the offending IP. It is extremely effective against brute-force attacks.

```
apt install fail2ban # /etc/fail2ban/jail.local [sshd] enabled = true port = ssh filter = sshd logpath = /var/log/auth.log maxretry = 5 bantime = 3600 # ban for 1 hour systemctl enable --now fail2ban fail2ban-client status sshd
```

■ VPNs and Encrypted Tunnels

- **WireGuard** — Modern, fast, and lean VPN built into the Linux kernel since 5.6. Recommended for new deployments.
- **OpenVPN** — Mature, widely supported, TLS-based. Good for compatibility with older clients.
- Always route administrative traffic (SSH, database connections) through a VPN rather than exposing those ports to the internet directly.

■ Port Scanning and Attack Surface Reduction

Regularly audit what is actually listening on your server. A port you didn't know was open is a vulnerability you didn't know you had.

```
# See what is listening ss -tlnp # Or with netstat netstat -tlnp # External port scan (run from another machine) nmap -sV -O your-server-ip
```

■ Minimal Attack Surface

*Every open port is a potential entry point. If a service is not needed, stop it and disable it.
systemctl disable --now . Regularly review services with systemctl list-units --type=service.*

8. Security Prep Before Going Online

A newly provisioned Linux server should never be connected to the public internet without first going through a hardening checklist. The window between 'online' and 'attacked' is shorter than most people realise — automated scanners probe every IP address continuously.

■ Pre-Launch Hardening Checklist

- Update all packages: **apt update && apt upgrade -y**
- Remove unnecessary software: **apt autoremove**. Fewer packages = fewer vulnerabilities.
- Enable and configure the firewall with default-deny inbound.
- Disable root SSH login. Create a non-root user with sudo access.
- Disable password-based SSH; use key authentication only.
- Enable fail2ban or equivalent brute-force protection.
- Configure automatic security updates.
- Enable audit logging (auditd).
- Run a port scan from another machine to verify only expected ports are open.
- Take an initial backup or snapshot before going live.

```
# Full update apt update && apt upgrade -y && apt autoremove -y # Disable
root SSH login # /etc/ssh/sshd_config: # PermitRootLogin no #
PasswordAuthentication no systemctl restart sshd # Verify no unexpected
open ports ss -tlnp
```

■ The Principle of Minimal Installation

If you are building a server, install only the software it needs for its specific role. A web server does not need a desktop environment, a compiler, or an email client. Each additional package is additional code — and additional risk.

■ *Cloud providers often offer pre-built images. Always verify these images against official checksums and review what software is pre-installed. Untrusted images can contain backdoors.*

9. Incident Response — During & After a Break-In

Despite best efforts, breaches happen. Having a clear, practised incident response plan dramatically reduces the damage. Panic is the enemy of good decision-making; a plan lets you act methodically under pressure.

■ Immediate Steps — Containment

- **Step 1 — Don't panic, don't reboot.** A reboot destroys volatile evidence (running processes, memory contents, network connections).

- **Step 2 — Isolate the system.** Disconnect from the network if possible — physically unplug the Ethernet cable. If virtual, snapshot the machine first, then isolate.
- **Step 3 — Preserve evidence.** Copy logs, capture running processes, and document network connections before doing anything else.
- **Step 4 — Change all passwords** immediately, from a clean machine — especially for accounts that have access to the compromised system.
- **Step 5 — Notify** your security team, management, and — if legally required — affected users or regulators.

■ Evidence Preservation Commands

```
# Capture running processes ps aux > /tmp/processes_$(date +%F).txt #
Capture network connections ss -tlnp > /tmp/netstat_$(date +%F).txt # List
recently modified files (last 24 hrs) find / -mtime -1 -type f 2>/dev/null
> /tmp/recent_files.txt # Copy important logs to external media cp -a
/var/log/ /mnt/usb_backup/logs/
```

■ Investigating Logs

```
# Authentication log — look for successful logins you don't recognise grep
'Accepted' /var/log/auth.log # Failed login attempts grep 'Failed password'
/var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn # Systemd
journal — last 500 lines journalctl -n 500 --no-pager # Kernel messages
dmesg | grep -i 'error\|warn\|attack'
```

■ Never trust the compromised system

Attackers often replace system binaries (ls, ps, netstat) with trojaned versions that hide their activity. For forensic investigation, boot from a trusted live USB or analyse a copy of the disk on a clean machine.

10. Closing the Hole & Damage Assessment

■ Identify and Patch the Vulnerability

Before bringing an affected system back online, you must understand **how** the attacker got in. Restoring from backup without fixing the root cause means getting breached again.

- Review logs to identify the initial point of entry (phishing, RCE exploit, brute-forced credential, etc.).
- Search for known CVEs related to the software versions on the compromised system.
- Apply patches, update software, or reconfigure services to close the identified vector.
- Audit for backdoors: new user accounts, authorised SSH keys, cron jobs, systemd services, and modified binaries.

```
# Look for unexpected cron jobs crontab -l ls -la /etc/cron*
```

```
/var/spool/cron/ # Look for new or modified systemd services find
/etc/systemd/system/ -newer /var/log/dpkg.log # Check for unexpected
authorised SSH keys find /home -name 'authorized_keys' -exec cat {} \; cat
/root/.ssh/authorized_keys # Check for new user accounts awk -F: '$3 >=
1000 {print $1, $3}' /etc/passwd
```

■ Assessing the Damage

A thorough damage assessment answers: *what was accessed, what was exfiltrated, what was modified, and what are the downstream consequences?*

- Determine if sensitive data (PII, credentials, financial records, intellectual property) was accessed or copied.
- Check integrity of binaries using package manager verification or known-good checksums.
- Assess reputational, legal, and financial impact — this feeds into whether breach notification is required.
- Document everything meticulously; this documentation may be required for legal proceedings or regulatory compliance.

```
# Debian/Ubuntu — verify installed package files against known checksums
debsums -c 2>/dev/null | head -50 # RHEL/CentOS — RPM verify rpm -Va
2>/dev/null | grep '^..5' | head -30
```

11. Backups — Your Safety Net

Backups are your last line of defence against ransomware, hardware failure, accidental deletion, and catastrophic incidents. The importance of backups cannot be overstated — and yet they are chronically under-tested. A backup you have never restored from is not a backup; it is a hope.

■ The 3-2-1 Rule

The industry-standard guideline for backup strategy:

- **3** — Keep at least three copies of your data.
- **2** — Store them on at least two different types of media (e.g., SSD and tape, or local disk and cloud).
- **1** — Keep at least one copy off-site (different geographic location or cloud provider).

■ Backup Tools

```
# rsync — efficient incremental backup rsync -avz --delete /var/www/
user@backup-server:/backups/www/ # tar — archive and compress tar -czf
backup_$(date +%F).tar.gz /etc /var/www /home # Automate with cron #
/etc/cron.d/daily-backup: 0 2 * * * root rsync -az /important/ /backup/ #
Restic — modern, encrypted, deduplicated backups restic -r /mnt/backup init
restic -r /mnt/backup backup /home
```

■ Backup Security Considerations

- Encrypt backups, especially off-site ones. A backup of your database in plain text is just another data breach vector.
- Restrict who can access backups — they often contain everything an attacker needs.
- Test restores regularly — at least quarterly, or whenever the backup software or data format changes.
- Backup configuration files (**/etc**) not just data. A fast recovery requires both.

■ Ransomware and the Air Gap

Ransomware encrypts your data and your connected backups. An air-gapped or write-once backup (e.g., WORM tape or S3 Object Lock) that the ransomware cannot reach is essential for true recovery.

12. Tracking Down the Intruder

After an incident is contained and the vulnerability patched, understanding the attacker's methods and goals — forensic analysis — helps prevent recurrence and may be legally required.

■ Log Analysis Tools

- **auditd** — Kernel-level audit daemon. Records file access, system calls, and command execution with timestamps and user IDs.
- **journalctl** — Query the systemd journal. Supports filtering by time, service, priority, and user.
- **fail2ban** — Its logs show which IPs were banned and for what activity.
- **grep / awk / sed** — Indispensable for cutting through large log files.
- **Elasticsearch + Kibana (ELK Stack)** — For centralised log management across multiple machines.

```
# journalctl examples journalctl -u sshd --since '2024-01-01' --until
'2024-01-02' journalctl -p err -b # all errors since last boot # auditd -
search for file access aureport -f -i | grep '/etc/shadow' # Find logins
from unexpected countries using IP geolocation grep 'Accepted password'
/var/log/auth.log | awk '{print $11}' | sort -u # Check bash history of all
users for f in /home/*/.bash_history /root/.bash_history; do echo "=== $f
==="; cat "$f" 2>/dev/null; done
```

■ Indicators of Compromise (IOCs)

- New or modified user accounts, especially those with UID 0 (root-level).
- Unexpected cron jobs, especially running as root or in **/etc/cron.d/**.
- Modified system binaries (check with **debsums/rpm -Va**).
- Outbound network connections to unusual IPs or on unusual ports.
- Large data transfers in outbound traffic — check firewall logs.

- Processes running from /tmp or /dev/shm — a classic malware staging area.

```
# Look for processes running from temp directories ls -la /proc/*/exe
2>/dev/null | grep -E 'tmp|shm' # Check outbound connections ss -tnp | grep
ESTABLISHED
```

13. Security Resources & Glossary

■ Trusted Security Resources

- **Linux Documentation Project (LDP)** — tldp.org — Original Linux HOWTO guides.
- **NIST Cybersecurity Framework** — nist.gov/cyberframework — Framework for managing cybersecurity risk.
- **CIS Benchmarks** — cisecurity.org — Detailed, consensus-based hardening guides for every major Linux distribution.
- **CERT Coordination Center** — cert.org — Incident response information and vulnerability disclosures.
- **Ubuntu Security Notices** — ubuntu.com/security/notices — CVE tracking for Ubuntu.
- **CVE Database** — cve.mitre.org — Authoritative registry of known vulnerabilities.
- **Arch Linux Security Advisories** — archlinux.org/security — Useful even if you don't use Arch.

■ Glossary

Term	Definition
Authentication	Verifying the identity of a user, process, or device.
Authorisation	Granting an authenticated entity permission to access a resource.
CVE	Common Vulnerabilities and Exposures — standardised identifier for known vulnerabilities.
DAC	Discretionary Access Control — file owner controls permissions.
DDoS	Distributed Denial of Service — flooding a target with traffic to make it unavailable.
Encryption	Converting plaintext data into unreadable ciphertext without the decryption key.
Firewall	System that filters network traffic based on rules.
IDS / IPS	Intrusion Detection / Prevention System — monitors for and optionally blocks attacks.
IOC	Indicator of Compromise — artefact suggesting a system has been breached.

LUKS	Linux Unified Key Setup — standard for disk encryption on Linux.
MAC	Mandatory Access Control — system-enforced policy overriding owner decision
PAM	Pluggable Authentication Modules — flexible authentication framework for Linux
Principle of Least Privilege	Grant only the minimum permissions required to perform a task.
RBAC	Role-Based Access Control — permissions tied to roles, not individuals.
SUID	Set User ID — execute a file with the file owner's privileges.
TLS	Transport Layer Security — cryptographic protocol securing network communication

Term	Definition
VPN	Virtual Private Network — encrypted tunnel over a public network.

14. FAQs & Conclusion

■ Frequently Asked Questions

Q: Why should I update Linux regularly?

A: Software updates include security patches that fix known vulnerabilities. An unpatched system is an open invitation — attackers actively scan for systems running vulnerable versions. Enable automatic security updates for low-effort protection.

Q: Why use strong passwords when I have a firewall?

A: A firewall controls network access, but once an attacker is inside — through a phishing email, a compromised account, or a legitimate user — the firewall is irrelevant. Strong passwords are a separate, complementary control.

Q: Is Linux more secure than Windows?

A: Linux has inherent advantages: fewer consumer-targeted malware campaigns, a strong permission model, and faster security patches. However, a poorly administered Linux server is far less secure than a well-managed Windows system. The human layer matters most.

Q: How do I know if my server has been compromised?

A: Indicators include: unexpected user accounts, unusual cron jobs, processes running from /tmp, unexplained outbound connections, high CPU/network usage, and entries in auth.log that you don't recognise. Regular log reviews and intrusion detection tools are your early warning system.

Q: Should I disable root login over SSH?

A: Absolutely yes. Root login over SSH means a successful brute-force attack immediately gives full system control. Create a regular user, grant sudo access, and disable root SSH. This single change eliminates a huge class of attacks.

Q: What is the single most important security measure?

A: If forced to choose one: keep software updated. The vast majority of successful attacks exploit known vulnerabilities for which patches already exist. Patching is unglamorous but consistently the highest-value security activity.

■ Conclusion

Linux security is a wide and deep field, but the fundamentals covered in these notes will take you a very long way. The most important insight is that security is not a destination — it is a discipline

that requires ongoing attention. Patch regularly, apply least privilege, monitor your systems, and plan for failure.

The attackers only need to succeed once. You need to stop them every time. Layered defences, good logging, tested backups, and a practiced incident response plan are what tip the balance back in the defender's favour.

Team Blue

- 1. ASAMPANA WISDOM APEGYINE**
- 2. ASUEM JOYCE**
- 3. LOTAA MOSES**
- 4. YAKUBU YUSSIF**
- 5. MAMUDU DREWAN**
- 6. BABA ALI**